

1. Common String Operations

Here are high-frequency string operations that often appear in competitive programming and technical interviews:

A. Concatenating Strings

You can efficiently build strings using the `+` and `+=` operators, or the `append()` method.

- **Warning:** You cannot concatenate two raw string literals together with `+`. At least one operand must be an `std::string` object.

```
1 #include <iostream>
2 #include <string>
3
4 int main() {
5     std::string first = "Hello";
6     std::string full = first + " World"; // OK
7     full += "!"; // Appends character
8
9     std::cout << full << "\n";
10 }
```

Output

Hello World!

B. Modifying Characters in Place

To process or alter every character (e.g., converting to uppercase or counting specific letters), a **range-based for loop** taking a reference (`auto&`) is the cleanest approach. Use functions from the `<cctype>` header like `std::toupper`, `std::tolower`, `std::isalpha`, or `std::isdigit`.

```
1 #include <cctype>
2 #include <iostream>
3 #include <string>
4
5 int main() {
6
7     std::string s = "India2026";
8     for (auto& ch : s) {
9         if (std::isalpha(ch)) {
10             ch = std::toupper(ch); // Converts letters to uppercase
11             std::cout << ch << " ";
12         }
13     }
14     std::cout << "\n";
15 }
```

Output

I N D I A

C. Erasing Substrings

Use the `erase(start_index, length)` function to remove characters from a string.

- **Warning:** Do not use `erase(i)` to remove a single character at index `i`. Providing only one argument removes *all* characters starting from `i` to the very end of the string. To remove one character, you must explicitly use `erase(i, 1)`.

```
1 #include <iostream>
2 #include <string>
3
4 int main() {
5     std::string phrase = "Assessment";
6     phrase.erase(0, 2); // Removes first 2 characters -> "essment"
7
8     std::cout << phrase << "\n";
9 }
```

Output

essment

D. Replacing Substrings

Use the `replace(start_index, length, new_string)` function to swap out a portion of a string. This is particularly useful when the replacement string is a different length than the original substring.

```
1 #include <iostream>
2 #include <string>
3
4 int main() {
5     std::string text = "I love C++";
6     // Replace 3 characters starting at index 7 ("C++") with "algorithms"
7     text.replace(7, 3, "algorithms");
8
9     std::cout << text << "\n";
10 }
```

Output

I love algorithms

E. Searching for Any Character from a Set using `std::string::find`

This `find` method is used to search for a single character, a C-style string, or another `std::string` within the text.

It returns the index (of type `size_t`) where the first match begins. If the target is not found, it returns a special static constant named `std::string::npos`.

Example:

```
1 #include <iostream>
2 #include <string>
3
4 int main() {
5     std::string sentence = "Manners maketh man";
6     std::string word = "man";
7
8     // Find the substring "man"
9     size_t index = sentence.find(word);
10
11     // Always check against std::string::npos to verify success
12     if (index != std::string::npos) {
13         std::cout << "Found at index: " << index << std::endl; // Outputs 15
14     }
15 }
```

Output

Found at index: 15

F Searching for Any Character from a Set using `std::string::find_first_not_of**`

If you need to split a string by **multiple potential delimiters** (e.g., spaces, commas, and periods), use `find_first_of()` to find the first occurrence of *any* character from a provided set. You can similarly use `find_first_not_of()` to find the first character that does *not* belong to a set.

```
1 #include <iostream>
2 #include <string>
3
4 int main() {
5     std::string text = "word1, word2. word3";
6     std::string delimiters = " ,."; // either space or , or .
7     size_t pos = text.find_first_of(delimiters); // Finds the comma
8
9     while (pos != std::string::npos) {
10         std::cout << "Found delimiter at position: " << pos << std::endl;
11         pos = text.find_first_of(delimiters, pos + 1); // Find next delimiter
12     }
13 }
```

Output

Found delimiter at position: 5
Found delimiter at position: 6

Found delimiter at position: 12
Found delimiter at position: 13

2. Examples

2.1. Finding and Extracting Substrings

To locate a substring or character within a string, use the `find()` function. It returns the index of the first match. If the target is not found, it returns the special constant `std::string::npos`. You can extract a portion of a string using `substr(start_index, length)`.

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string text = "Amazon software engineering internship";
6      std::string target = "engineering";
7
8      // Find the starting index of the substring
9      size_t pos = text.find(target);
10
11     // Always check against std::string::npos to verify it was found
12     if (pos != std::string::npos) {
13         // Extract 11 characters starting from 'pos'
14         std::string word = text.substr(pos, target.length());
15         std::cout << "Found: " << word << " at index " << pos << "\n";
16     }
17
18     // To search backward from the end of the string, use rfind()
19     size_t last_e = text.rfind('e'); // Finds the last 'e'
20     if (last_e != std::string::npos) {
21         std::cout << "Last occurrence of 'e' is at index " << last_e << "\n";
22     }
23 }
```

Output

Found: engineering at index 16
Last occurrence of 'e' is at index 31

Pro-tip: Remember that the second argument to `substr()` is the **length** of the substring to extract, not the ending index.

2.2. Reading a Line and Parsing Delimited Fields (CSV Style)

2.2.1. Version 1

Online assessments often provide input as a single comma-separated line. To handle this, read the full line using `std::getline()`, then wrap it in an `std::stringstream` (from the `<sstream>` header) to treat the string as an input stream.

You can use `std::getline` with a third argument to specify a custom delimiter (like a comma `,`). To convert string tokens into numeric types, use the standard library functions `std::stoi()` (string to int) and `std::stol()` / `std::stoll()` (string to long / long long).

```
1  #include <iostream>
2  #include <string>
3  #include <sstream>
4
5  struct Transaction {
6      std::string account;
7      int timestamp;
8      std::string currency;
9      long amount;
10 };
11
12 int main() {
13     std::string input_line = "acct_123, 1, usd, 1000";
14     std::stringstream ss(input_line);
15     std::string token;
16     Transaction t;
17
18     // 1. Extract Account
19     std::getline(ss, t.account, ',');
20
21     // 2. Extract Timestamp (read into string, convert to int)
22     std::getline(ss, token, ',');
23     t.timestamp = std::stoi(token);
24
25     // 3. Extract Currency
26     std::getline(ss, t.currency, ',');
27
28     // 4. Extract Amount (read into string, convert to long)
29     std::getline(ss, token, ',');
30     t.amount = std::stol(token);
31
32     // Note: std::stoi and std::stol automatically ignore leading whitespace!
```

```

33
34 // Output the parsed transaction
35 std::cout << "Account: " << t.account << std::endl;
36 std::cout << "Timestamp: " << t.timestamp << std::endl;
37 std::cout << "Currency: " << t.currency << std::endl;
38 std::cout << "Amount: " << t.amount << std::endl;
39 }

```

Output

```

Account: acct_123
Timestamp: 1
Currency:  usd
Amount: 1000

```

Note: While `std::stoi()` automatically ignores leading whitespace, extracting the string `t.currency` directly with `getline` might leave a leading space (e.g., " usd"). If exact string matching is required, you may need to manually trim leading spaces.

Explanation on `getline`

In the previous example, we used `std::getline` to parse comma-separated fields from a `std::stringstream`. Here is exactly how it works under the hood:

The `std::getline()` function is defined in the `<string>` header and takes up to three parameters: a reference to an input stream, a reference to a destination string, and an optional delimiter character.

When you write `std::getline(ss, t.account, ',')`, the following sequence of events occurs:

1. **Reading Characters:** The function reads characters one by one from the input stream (`ss`) and stores them into the destination string (`t.account`).
2. **Stopping at the Delimiter:** It continues reading until it encounters the exact delimiter character you specified as the third argument (in this case, the comma `,`).
3. **Discarding the Delimiter:** When it hits the delimiter, `getline` stops reading. It extracts the delimiter from the stream and **discards it**. The comma is *not* appended to the resulting string.
4. **Preparing for the Next Read:** Because the comma was extracted and thrown away, the stream's internal marker is now positioned perfectly at the start of the next field (e.g., the space or the `'1'`), ready for the next extraction operation.

The Return Value and Loops The `getline` function returns the input stream that was passed to it. Because input streams can be implicitly evaluated as a boolean condition, you will often see `getline` used directly inside a `while` loop condition. The condition will evaluate to true as long as the read was successful, and false if the stream hits the end-of-file or encounters an error.

The Default Behavior If you omit the third argument, as in `std::getline(cin, line)`, the function automatically defaults to using the newline character (`'\n'`) as the delimiter. It will read the entire line of text, discard the newline character at the end, and return the string without it.

2.2.2. Parsing Delimited Fields in multiple lines: Processes below kind of input from stdin.

```
2
2
acct_123, 1, usd, 1000
acct_456, 2, usd, 2000
3
acct_143, 2, usd, 2000
acct_245, 2, usd, 2000
acct_156, 2, int, 50000
```

The input consists of an initial integer representing the total number of groups (2 in this case). Then, for each group, there is an integer representing the number of transactions in that group, followed by the actual comma-separated transaction lines.

Create vector of total 5 Transaction.

When processing this, the biggest “gotcha” is mixing `std::cin >> variable` (to read the counts) with `std::getline()` (to read the full text lines). When `std::cin >>` reads an integer, it leaves the newline character (`\n`) in the input stream. If you immediately call `getline()`, it will instantly read that leftover newline and return an empty string.

To fix this, we can use the `std::ws` stream manipulator to consume and discard any leading whitespace (including newlines) right before we start reading the lines.

Here is the complete example demonstrating how to parse your exact input into a `std::vector<Transaction>`:

```
1  #include <iostream>
2  #include <string>
3  #include <sstream>
4  #include <vector>
5
6  struct Transaction {
7      std::string account;
8      int timestamp;
9      std::string currency;
10     long amount;
11 };
12
```

```

13  int main() {
14      int num_groups;
15      // Read the total number of groups (e.g., 2)
16      if (!(std::cin >> num_groups)) return 0;
17
18      std::vector<Transaction> all_transactions; // Vector to hold all 5 transactions
19
20      // Loop over each group
21      for (int i = 0; i < num_groups; ++i) {
22          int num_transactions;
23          // Read how many transactions are in this group (e.g., 2 then 3)
24          std::cin >> num_transactions;
25
26          // CRITICAL: Consume the trailing newline left by std::cin >> num_transactions
27          // so that getline() doesn't immediately read an empty line.
28          std::cin >> std::ws;
29
30          // Read the specified number of transactions
31          for (int j = 0; j < num_transactions; ++j) {
32              std::string line;
33              std::getline(std::cin, line);
34
35              std::stringstream ss(line);
36              std::string token;
37              Transaction t;
38
39              // Parse the comma-separated fields
40              // account
41              std::getline(ss, t.account, ',');
42
43              // timestamp
44              std::getline(ss, token, ',');
45              t.timestamp = std::stoi(token);
46
47              // currency
48              // ***** handle leading space *****
49              // Just as we used std::ws to consume the lingering newline
50              // character in the std::cin stream, we can also apply it directly
51              // to our std::stringstream immediately before calling std::getline.
52              // This will quietly consume any leading whitespace (like spaces or
53              // tabs) so getline only reads the actual text.
54
55              ss >> std::ws; // Consumes the leading space(s) in the stream
56              std::getline(ss, t.currency, ',');
57
58              std::getline(ss, token, ',');

```



```

59         t.amount = std::stol(token);
60
61         // Append the parsed transaction to our master vector
62         all_transactions.push_back(t);
63     }
64 }
65
66 // Output to verify we successfully stored all 5 transactions
67 std::cout << "Successfully parsed " << all_transactions.size() << " transactions.\n";
68 for (const auto& t : all_transactions) {
69     std::cout << "Account: " << t.account << ", Amount: " << t.amount << "\n";
70 }
71
72 return 0;
73 }

```

Output

```

$ ./a.out < cpp_strings_input.txt
Successfully parsed 5 transactions.
Account: acct_123, Amount: 1000
Account: acct_456, Amount: 2000
Account: acct_143, Amount: 2000
Account: acct_245, Amount: 2000
Account: acct_156, Amount: 50000

```

where `cpp_strings_input.txt` has below content:

```

2
2
acct_123, 1, usd, 1000
acct_456, 2, usd, 2000
3
acct_143, 2, usd, 2000
acct_245, 2, usd, 2000
acct_156, 2, int, 50000

```

Key Takeaways for this Pattern:

1. **std::vector for dynamic storage:** Because we don't necessarily know the total number of elements upfront, we start with an empty `std::vector` and use `push_back()` to dynamically append elements as we read them.
2. **std::cin >> std::ws:** Always use this when transitioning from reading formatted data (like `int` or `double` using `>>`) to reading unformatted data (like full strings using `getline`).
3. **Nested loops:** The outer loop handles the “groups,” and the inner loop processes the exact number of rows declared for that specific group.

Note: In the C++ Standard Library, the normal string input operator (>>) automatically skips leading whitespace. However, because we are using `getline()` to explicitly handle the comma delimiters, we bypass that automatic skipping and must handle the spaces ourselves.

Alternative Method 2: Trimming the String After Extraction If you want to read the exact string first and then manually trim the spaces, you can use the string's `find_first_not_of()` and `erase()` or `substr()` methods.

This is particularly useful if you want to explicitly check for formatting irregularities in the data before altering it.

```
// 3. Extract Currency
std::getline(ss, t.currency, ',');

// Find the index of the first character that is NOT a space
size_t startpos = t.currency.find_first_not_of(" \t");

// If it found a valid character, erase everything before it
if (startpos != std::string::npos) {
    t.currency.erase(0, startpos);
}
```